mWater → ThingsBoard Data Transfer

Setup, daily use and repair guide — INOWAS dashboards

This document explains how measurement data is transferred from mWater into ThingsBoard (`dashboards.inowas.org`), how to add a new dataset, and how to repair the system if something breaks. It is written so that a colleague who has never touched the setup can keep it running.

What the system does

Data is not exported and uploaded by hand. Instead, a rule chain inside ThingsBoard downloads the mWater export on a timer and writes it into devices as time series. The full path:

mWater JSON export link → **Rule chain** mWater Sync (auto-create) → **Devices** one per measurement point → **Dashboards** charts / tables

The four components

Component	What it is	Where to find it
mWater config	A normal device used only as a shelf. It holds the list of datasets in a server attribute called `datasets`. No telemetry is ever written to it.	Entities → Devices
datasets attribute	A JSON array: one entry per dataset (export URL, target device, date column, point column, sync flag).	mWater config → Attributes → Server scope
mWater Sync (auto-create)	The rule chain. On a timer it reads the list, downloads each export, converts the dates and writes telemetry. It creates missing devices by itself.	Entities → Rule chains
The widget	A small editor for the `datasets` attribute. It does not move any data itself.	Resources → Widgets library → Custom tools

> **Rule of thumb:** the widget only edits a list. All the real work happens in the rule chain. If data stops arriving, the rule chain is where you look — not the widget.

Requirements

- **Tenant administrator rights** in ThingsBoard. Without them the widget cannot read or write the list, and the rule chain cannot create devices.
- The export link from mWater must be in **JSON** format (not CSV) and reachable from the internet.
- No access token has to be copied anywhere: the widget uses your browser session and refreshes it automatically.

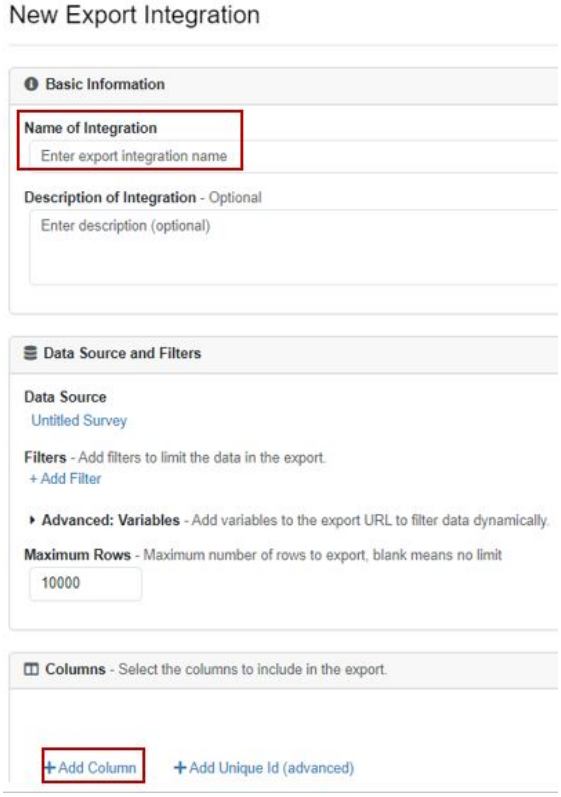
1. Create the export link in mWater

Every dataset needs its own export link. This is done once per dataset.

1. In mWater, open **Manage** → **Integrations**, then click **+ New Export Integration** (top right).

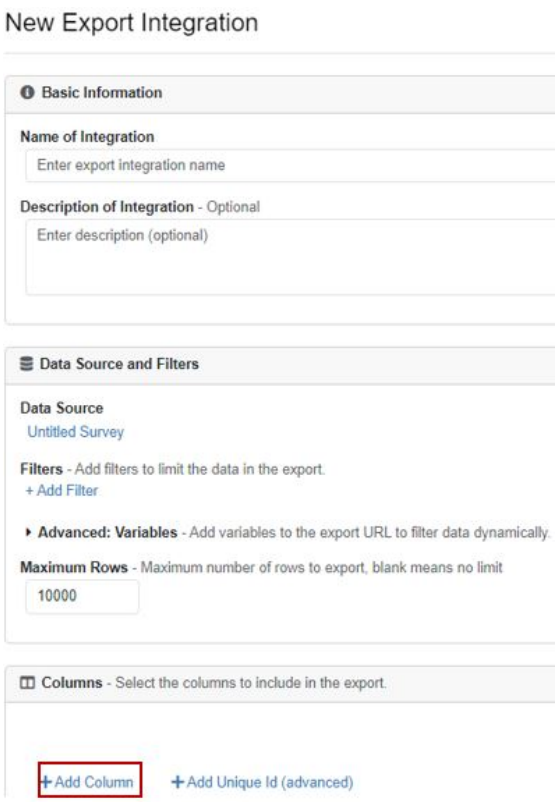
Manage → Integrations → New Export Integration

2. Give the integration a **name**, then add the columns you need with **Add Column**: the date column, the point column (only if the dataset contains several measurement points) and the measured parameters.



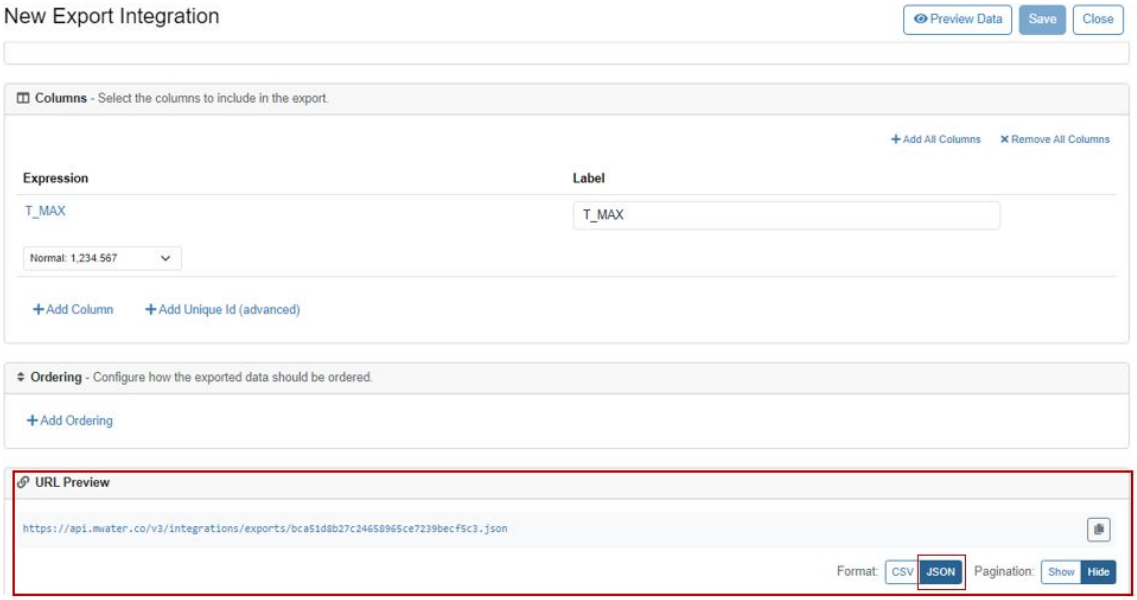
Name of Integration and Add Column

3. Check **Maximum Rows**. Leave it **empty** to export the full history — a value here silently truncates the export.



Maximum Rows: leave empty for the full history

4. Scroll to **URL Preview**, switch the format to **JSON**, and copy the link. Then **Save** the integration.



URL Preview → format JSON → copy the link

> Paste the link into a browser to check it: it must show a JSON array (text starting with a square bracket). If a file downloads instead, or you see a comma-separated table, the format is still CSV and the rule chain will not be able to read it.

2. Add a dataset with the widget

The widget writes your entry into the `datasets` list. The rule chain picks it up on its next run.

Choosing the mode

- **Single device** — the dataset contains one measurement point (or you want everything in one device). Fill in **Device name**. Leave the point column empty.
- **Split by point** — the dataset contains many measurement points in one table. Fill in **Point column** with the name of the column that holds the point name (for example `Name`). Each point then gets its own device, named after the point. **Prefix** is optional and is put in front of every device name.

Fields

Field	Meaning
mWater JSON export URL	The link from section 1.
Date column	Name of the column holding the date, e.g. `Date`, `Time` or `submitted on`. Both Excel serial numbers and normal date/time text are understood.
Point column	Only in *Split by point*: the column that names the measurement point.
Device name / Prefix	Target device (single mode) or optional name prefix (split mode).
sync	On: the dataset is transferred on every run. Off: the rule chain skips it.

Buttons

- **Add dataset** — adds the entry to the table on screen. Nothing is stored yet.
- **Save** — writes the list to ThingsBoard. **Changes only take effect after Save.**
- **Reload** — re-reads the stored list and discards unsaved edits.

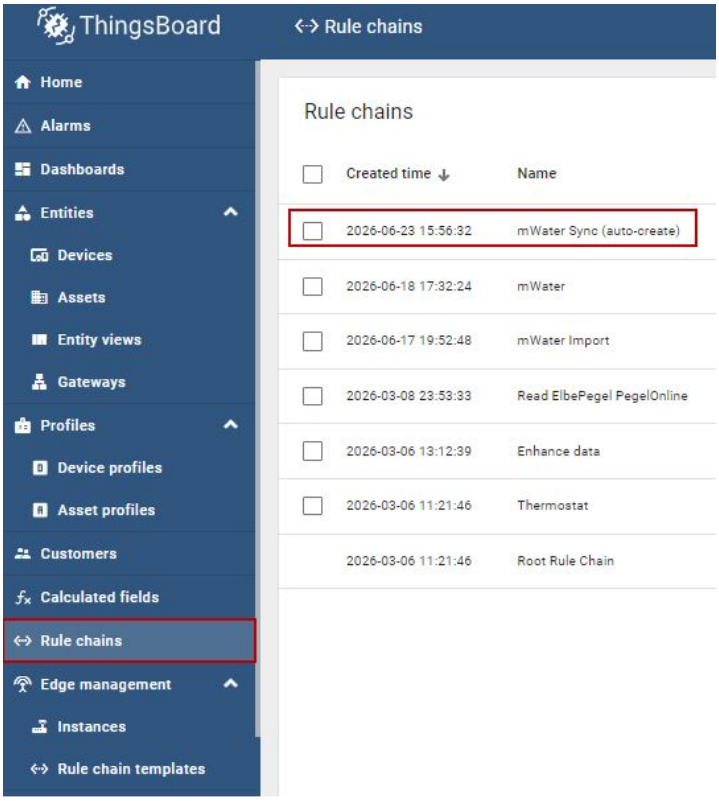
> **Historical data:** a dataset that never changes (for example a 30-year climate series) only has to be transferred once. When the data has arrived, switch **sync** off and press Save. The rule chain then stops re-uploading thousands of points on every run. Switch it back on when new rows are added in mWater.

Checking that it worked

1. Wait for one run of the rule chain (see the Tick period in section 3).
2. Open **Entities** → **Devices**. In split mode the point devices appear automatically, with device type `mWater point`.
3. Open one device → **Latest telemetry**. The measured parameters must be listed there.
4. For the history, add a chart and set the time window wide enough to cover the dates in the data. The default window only shows the last few hours, which looks empty for historical series.

3. The rule chain

The chain is called **mWater Sync (auto-create)** and lives under **Entities** → **Rule chains**. It runs on the server, so nothing has to be open in a browser.



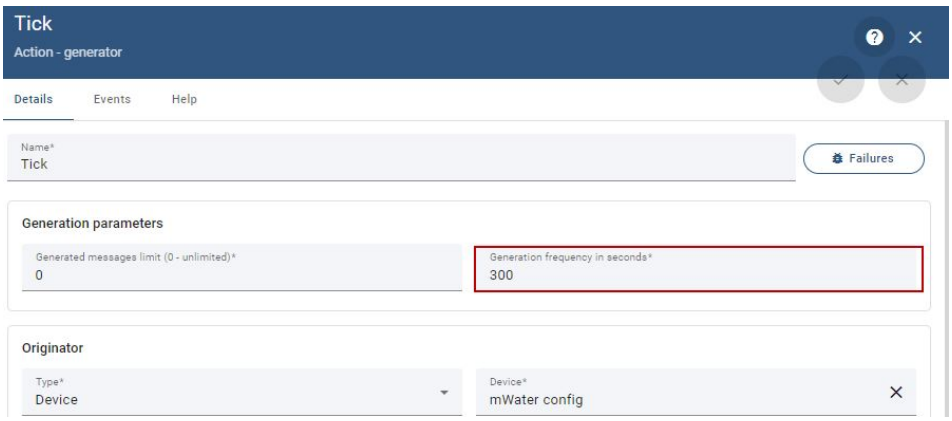
Entities → Rule chains → mWater Sync (auto-create)

What each node does

Node	Function
Tick	Timer. Starts one run. Its *originator* must be the **mWater config** device, because the next node reads the list from the originator.
Load datasets	Reads the server attribute `datasets` from the originator.
Split datasets	Turns the list into one message per dataset and skips entries whose sync flag is off.
Fetch mWater	Downloads the export URL (GET).
Split rows	Splits the downloaded array into single rows. This node is required: script nodes reject any input larger than 100,000 characters.
Transform	Per row: converts the date into a timestamp, decides the target device name, and builds the telemetry values.
Has device	Filter. Drops rows with an empty or unwanted point name, so no junk devices are created.
Create + route to device	Creates the target device if it does not exist yet, and sends the row to it.
save	Writes the time series.

Changing how often the transfer runs

Open the **Tick** node and edit **Generation frequency in seconds**. Use `86400` (once a day) for normal operation, or a smaller value while testing. Do not go below about two or three minutes: each run re-uploads all rows of every active dataset, and runs would start to overlap.



Tick node: frequency and originator (mWater config)

After changing anything in a node, apply the node and then **save the rule chain**.

4. Troubleshooting

Almost every problem is found the same way: run the chain with debug on and look at the **Events** tab of each node, from left to right. The first node without outgoing events is where it stops.

How to read the events

1. Open the rule chain, double-click a node, switch debug from *Failures* to **all messages**, and save the chain.
2. Wait for one Tick period.
3. Open the **Events** tab of each node in order: Tick, Load datasets, Split datasets, Fetch mWater, Split rows, Transform, Has device, Create + route, save.
4. In a row, the **Data** column shows the message content, **Metadata** shows values such as the URL and the target device, and **Error** shows the failure text.

> Full debug logging switches itself off after about 15 minutes. Empty event lists usually mean debug expired — not that the chain has stopped. Turn debug on again right before testing, and turn it off afterwards.

Common problems

Symptom	Cause and fix
Widget says *Save failed: Token has expired*	The browser session expired. Reload the dashboard page and press Save again. If it repeats, log out and back in.
Widget saves, but no data appears	Check the export link in a browser first. Then check the Events of **Fetch mWater**: if there is no response, the URL is wrong or not public. A local file path (D:\...) can never work — the chain runs on the server.
Script input arguments exceed maximum allowed total args size	The **Split rows** node is missing or bypassed. The whole export is being handed to a script node. Reconnect Fetch mWater → Split rows → Transform.
Data lands with wrong dates	Wrong **Date column** in the dataset entry. Correct it in the widget and save. Note that `submitted on` is the upload time, which is not always the measurement date.
Only part of the history arrives	The mWater export is truncated or paginated. Clear **Maximum Rows** in the integration and check that pagination is hidden in the URL preview.
Junk devices appear	Rows with an unusual point name. Extend the skip list in the **Transform** node (see appendix).
Nothing runs at all	Check the **Tick** node: its originator must point at an existing **mWater config** device. If that device was deleted or recreated, see section 5.

5. Repairing the system

If the mWater config device was deleted

This is the only part that is not rebuilt automatically. Everything else (point devices and their data) is recreated by the rule chain from mWater.

1. Create a device named **mWater config** (Entities → Devices → Add device). The device type does not matter.
2. Copy its **id** from the device card — a new device always gets a new id.
3. Open the rule chain, edit the **Tick** node, and set *Originator* to Device → **mWater config** (pick it from the list). Apply and save the chain.
4. Open the widget in the widgets library, replace the id in the first line of the JavaScript tab (`CONFIG_DEVICE_ID`), press Run, then Save. If the widget is already on a dashboard, remove it and add it again.
5. Enter the datasets again in the widget and press Save. The attribute is created automatically.
6. Wait for one or two runs: the point devices reappear and the history is re-uploaded from mWater.

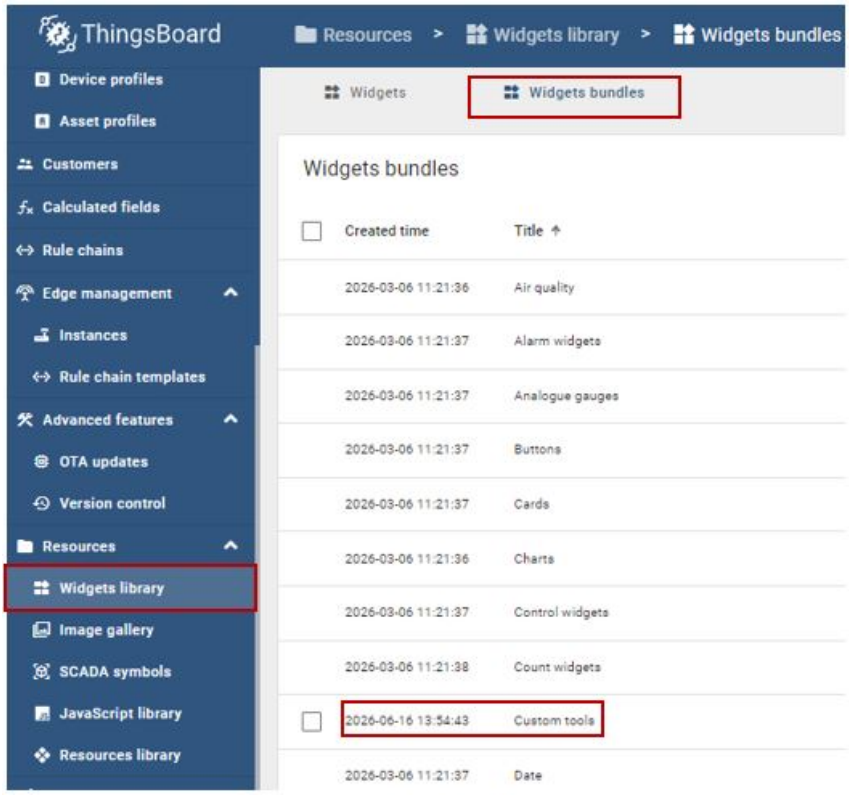
> **Keep a backup of the list.** The `datasets` attribute is the only piece of information that cannot be recovered from mWater. Copy its text (plain JSON) into a file or a wiki page whenever it changes. Recovery then takes two minutes instead of an afternoon.

If the rule chain was deleted or broken

Rule chains can be exported and imported as JSON files: open **Entities** → **Rule chains**, use the export action to save a copy, and the import button to restore one. Keep an exported copy of **mWater Sync (auto-create)** next to the dataset list. If no export exists, the chain can be rebuilt node by node using the table in section 3 and the scripts in the appendix.

If the widget stops loading

- The widget lives in **Resources** → **Widgets library** → **Widgets bundles** → **Custom tools**.
- Its code is split over three tabs: **HTML** (layout only), **CSS** (styling), **JavaScript** (logic). Style rules must stay in the CSS tab — curly brackets inside the HTML tab break the template compiler.
- After editing, press **Run** to compile and then **Save**. A widget already placed on a dashboard keeps an older copy: remove it from the dashboard and add it again to pick up changes.



Resources → Widgets library → Widgets bundles → Custom tools

Appendix: node scripts

These are the scripts used inside the rule chain. They are only needed if a node has to be rebuilt. All of them are JavaScript (set *Script language* to JS in the node).

Split datasets

Turns the stored list into one message per dataset and skips disabled entries.

```
var raw = metadata.datasets || metadata.ss_datasets || msg.datasets;
var list = (typeof raw === "string") ? JSON.parse(raw) : raw;
var out = [];
for (var i = 0; i < list.length; i++) {
    var d = list[i];
    if (d.enabled === false) { continue; }
    out.push({
        msg: {},
        metadata: { url: d.url, device: d.device || "",
                    dateField: d.dateField || "Date",
                    pointField: d.pointField || "" },
        msgType: "MWATER_FETCH"
    });
}
return out;
```

Transform

Runs on a single row: builds the timestamp, the target device name and the telemetry values. Add unwanted point names to `SKIP_POINTS`.

```
var dateField  = metadata.dateField  || "Date";
var pointField = metadata.pointField || "";
var base       = metadata.device || "";

var SKIP_POINTS = { "": true, "Other (please specify)": true };

var target;
if (pointField) {
    var point = (msg[pointField] != null) ? ("" + msg[pointField]).trim() : "";
    if (SKIP_POINTS[point]) { target = ""; }
    else { target = base ? (base + " - " + point) : point; }
} else {
    target = base;
}

var raw = msg[dateField];
var ts = null;
if (typeof raw === "number") {
    if (raw < 100000)    { ts = (raw - 25569) * 86400000; }
    else if (raw < 1e11) { ts = raw * 1000; }
    else                 { ts = raw; }
} else if (typeof raw === "string" && raw !== "") {
    var m = raw.match(/^(\d{4})-(\d{2})-(\d{2})(?:[ T](\d{2}):(\d{2})(?::(\d{2}))?)?/);
    if (m) { ts = Date.UTC(+m[1], +m[2]-1, +m[3], +(m[4]||0), +(m[5]||0), +(m[6]||0)); }
    else { var p = Date.parse(raw); if (!isNaN(p)) { ts = p; } }
}

var EXCLUDE = { "code": true, "submitted on": true, "submitted": true,
                "submitted_on": true, "Location": true };
EXCLUDE[dateField] = true;
if (pointField) { EXCLUDE[pointField] = true; }

var values = {};
for (var k in msg) {
    if (EXCLUDE[k]) { continue; }
    var v = msg[k];
    if (v === null || v === "") { continue; }
    values[k] = v;
}

var md = { device: target };
if (ts !== null) { md.ts = "" + ts; }
```

```
return { msg: values, metadata: md, msgType: "POST_TELEMETRY_REQUEST" };
```

Has device (filter)

Passes only rows that have a target device. The *True* output goes to the next node.

```
return metadata.device !== "" && metadata.device != null;
```

Create + route to device

This node is configured in the interface, not by script. The settings that matter: entity type **Device**, name pattern `${device}`, **create entity if it does not exist** enabled, and **change originator to the related entity** enabled.

Example of a stored dataset list

This is what the `datasets` attribute looks like. The first entry splits one export into one device per measurement point; the second sends everything into a single device.

```
[
  { "url": "https://api.mwater.co/v3/integrations/exports/....json",
    "device": "",
    "dateField": "Time",
    "pointField": "Name",
    "enabled": true },
  { "url": "https://api.mwater.co/v3/integrations/exports/....json",
    "device": "Nukus climate",
    "dateField": "Date",
    "pointField": "",
    "enabled": false }
]
```